
Software Requirements Specification

for

Pothik Bondhu

Hajj & Umrah Portal

Prepared by ABDULLAH AL MAHMUD EMON

Date : 30 - 04 - 2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose	1
1.2 Motivation	1
1.3 Objectives	1
1.4 Product Scope.....	1
1.5 References	1
2. Overall Description	1
2.1 Product Perspective	1
2.2 Product Functions.....	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment	2
2.5 Design and Implementation Constraints.....	3
2.6 User Documentation	3
2.7 Assumptions and Dependencies	3
3. External Interface Requirements	3
3.1 User Interfaces	3
3.2 Hardware Interfaces.....	5
3.3 Software Interfaces	5
3.4 Communications Interfaces	6
4. System Features	6
4.1 System Feature 1	6
4.2 System Feature 2 (and so on)	6
5. Other Nonfunctional Requirements	6
5.1 Performance Requirements.....	6
5.2 Safety Requirements.....	6
5.3 Security Requirements.....	6
5.4 Software Quality Attributes.....	7
5.5 Business Rules.....	7
6. Other Requirements	7
Appendix A: Glossary.....	7
Appendix B: Analysis Models	7
Appendix C: To Be Determined List.....	7

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The primary objective of the Pothik Bondhu system is to digitize and streamline the operations of religious travel agencies managing Hajj and Umrah pilgrimages. Currently, many agencies rely on inefficient manual paperwork, phone conversations, and decentralized physical records to manage their workflows. This system replaces those traditional methods with a centralized, organized digital platform, ensuring operational efficiency and a seamless tracking experience for both agency staff and clients.

1.2 Motivation

Pothik Bondhu was created to fix the messy, outdated ways Hajj and Umrah travel agencies currently operate. Instead of relying on a confusing mix of paper files, WhatsApp messages, and spreadsheets that often cause lost documents and delayed government paperwork, this platform brings all those scattered tasks together into one secure, organized, and easy-to-use digital system that allows both users and administrators to easily track a pilgrim's profile and progress.

1.3 Objectives

*Centralize Agency Management
Automate Compliance Reporting
Implement a Hybrid Workflow
Enhance Client Transparency*

1.4 Product Scope

The system will handle user authentication, package browsing, branch geolocation, hybrid document submission and verification workflows, milestone payment tracking (simulated online and manual offline logging), and data export functionalities for government compliance.

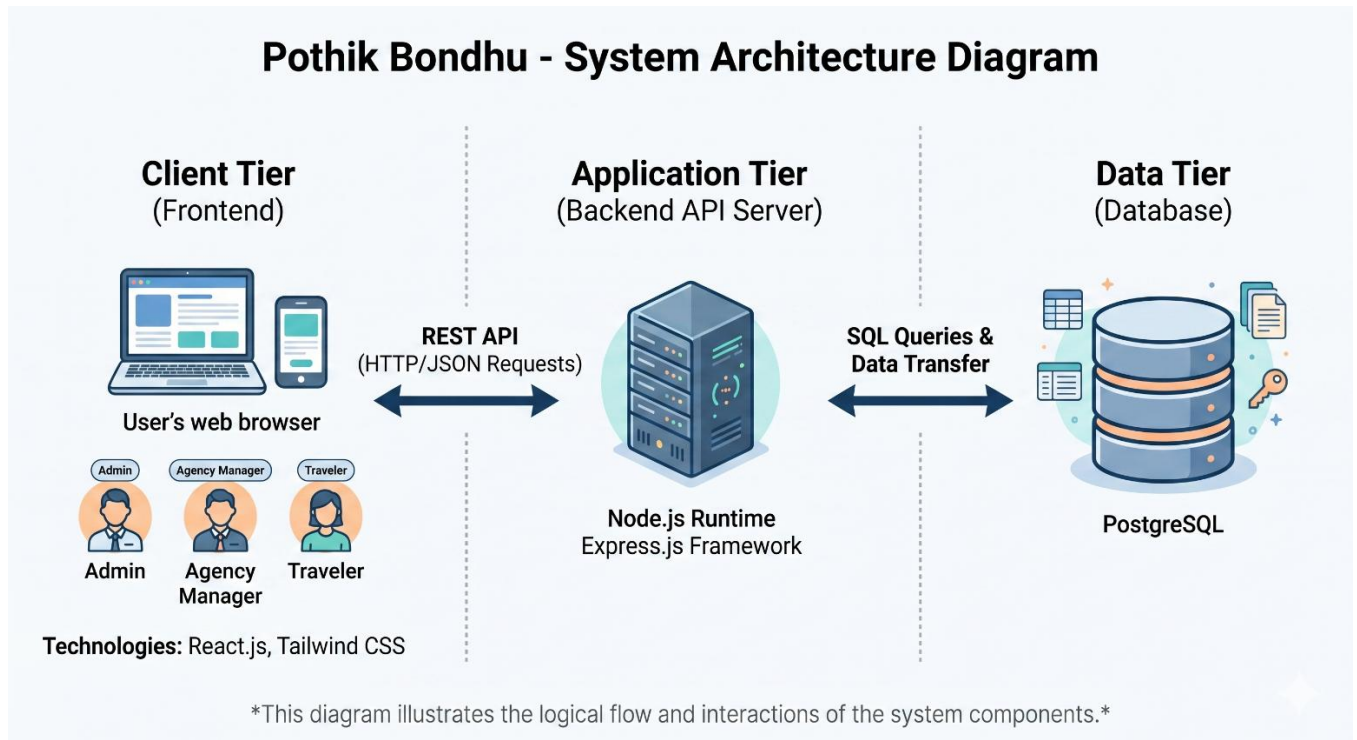
1.5 References

<>

2. Overall Description

2.1 Product Perspective

Pothik Bondhu is an independent, web-based travel agency management system. It operates as a centralized platform designed to streamline operations and logistics between travel agencies, their



regional branches, and travelers. The system follows a standard client-server architecture, utilizing a web-based frontend that communicates securely with a backend API and a centralized relational database.

2.2 Product Functions

The primary functions of the system include:

- Centralized management of travel agencies and their respective branches.
- Traveler coordination and booking record management.
- Role-based dashboards providing specific views and controls for administrators, agency managers, and users.
- Real-time data handling for system updates and user communication.

2.3 User Classes and Characteristics

The system is designed for three main types of users:

- **System Administrator:** Has full access to the system to onboard new agencies, manage global settings, and oversee system health. Characteristics: Moderate technical expertise.
- **Agency/Branch Manager:** Uses the system for daily operations to coordinate travelers, update branch-specific records, and manage logistical data. Characteristics: Basic computer and internet navigation skills.
- **Traveler/Customer:** Uses the frontend interface to view agency offerings, track itineraries, and communicate. Characteristics: General public internet user.

2.4 Operating Environment

The system will operate in the following environments:

- **Client-Side:** Modern web browsers (Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge) running on desktop, tablet, or mobile operating systems.
- **Server-Side:** Node.js runtime environment.
- **Database:** PostgreSQL relational database.

2.5 Design and Implementation Constraints

The development of the system is bound by the following constraints:

- *The frontend must be implemented using React and styled with CSS.*
- *The backend application must be built using Node.js and the Express framework.*
- *System data must be managed using a PostgreSQL database.*
- *Development must be completed within the specific timeframe allotted for the academic semester project.*

2.6 User Documentation

A comprehensive technical README file will be provided, detailing environment setup, database configuration, and local deployment instructions for developers and evaluators. For end-users, the user interface is designed to be self-explanatory, utilizing standard web conventions, contextual tooltips, and clear navigation menus to eliminate the need for extensive user manuals.

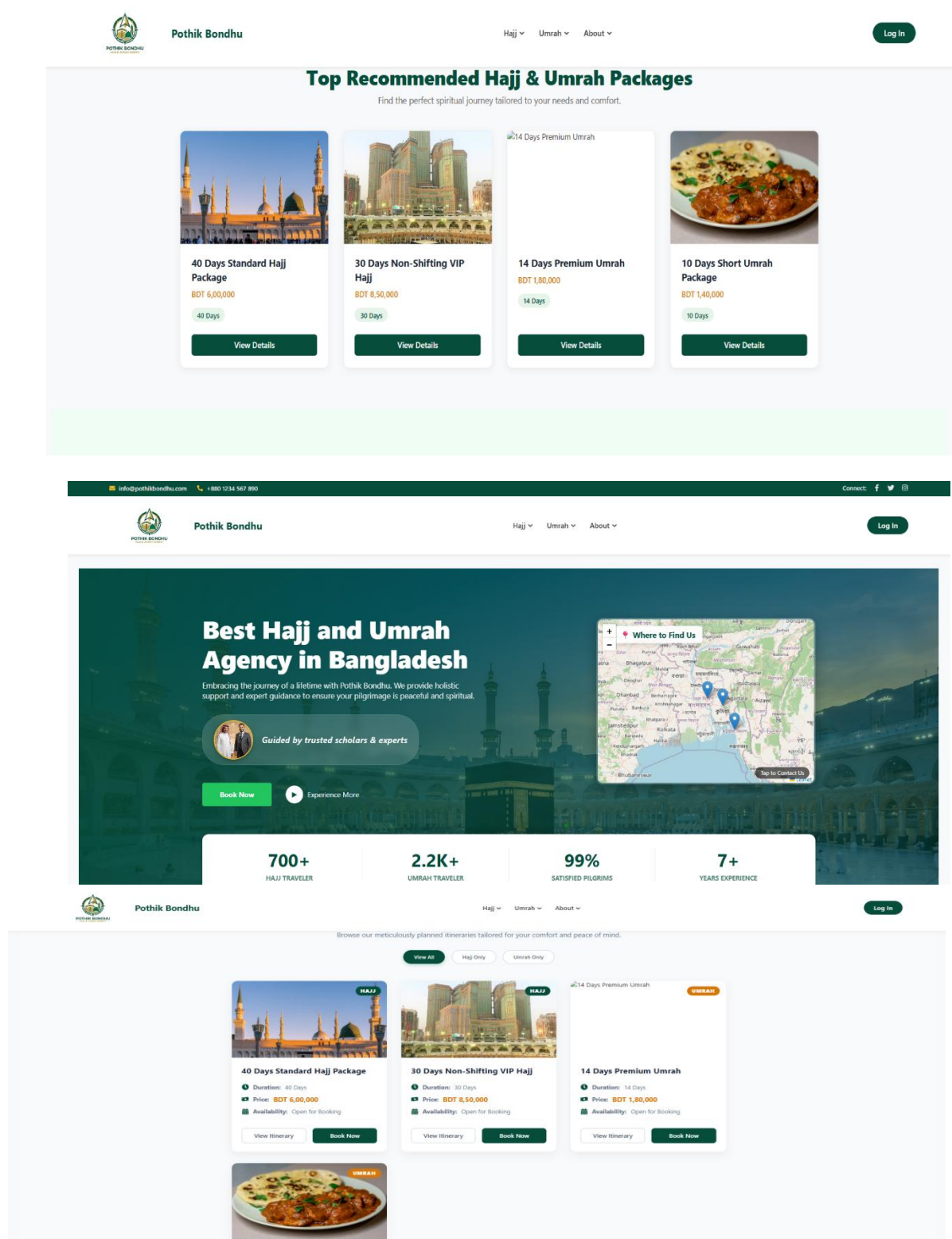
2.7 Assumptions and Dependencies

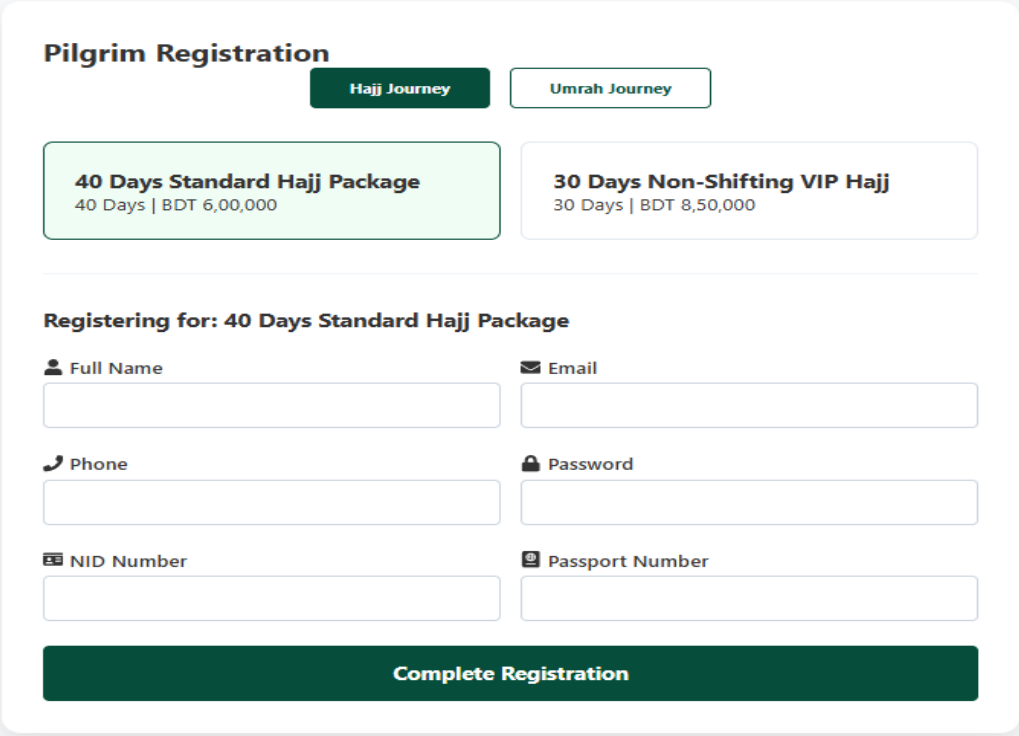
- **Assumptions:** It is assumed that all users will access the system via a device with a stable internet connection and a modern web browser with JavaScript enabled.
- **Dependencies:** The system relies on continuous support for the open-source technologies used in the PERN stack. It also depends on third-party cloud hosting services for database hosting and frontend deployment, as well as various Node Package Manager (NPM) libraries for core functionalities.

3. External Interface Requirements

3.1 User Interfaces

The user interface of Pothik Bondhu is designed to be responsive, intuitive, and accessible across various devices. It is built using React.js and styled with Tailwind CSS to ensure a modern, clean aesthetic.





The image shows a web form titled "Pilgrim Registration". At the top, there are two buttons: "Hajj Journey" (highlighted in green) and "Umrah Journey". Below these are two package options: "40 Days Standard Hajj Package" (40 Days | BDT 6,00,000) and "30 Days Non-Shifting VIP Hajj" (30 Days | BDT 8,50,000). The form is currently set for the "40 Days Standard Hajj Package". The registration fields include: Full Name, Email, Phone, Password, NID Number, and Passport Number. A green "Complete Registration" button is at the bottom.

Pilgrim Registration

Hajj Journey Umrah Journey

40 Days Standard Hajj Package
40 Days | BDT 6,00,000

30 Days Non-Shifting VIP Hajj
30 Days | BDT 8,50,000

Registering for: 40 Days Standard Hajj Package

Full Name Email

Phone Password

NID Number Passport Number

Complete Registration

3.2 Hardware Interfaces

As a web-based application, Pothik Bondhu does not require any specialized hardware interfaces (such as barcode scanners or custom sensors).

- **Client Devices:** The system interfaces with standard consumer hardware, including desktop computers, laptops, tablets, and smartphones.
- **Server Hardware:** The application will run on standard cloud-based server hardware capable of hosting Node.js applications and PostgreSQL databases.

3.3 Software Interfaces

The system relies on the following software components to function:

- **Operating Systems:** The client application is OS-agnostic and runs within standard web browsers. The backend server will be hosted on a Linux or Windows-based cloud server.
- **Database:** The system interfaces with a **PostgreSQL** database to store, retrieve, and update user profiles, agency records, and booking data.
- **Application Programming Interface (API):** The React frontend communicates with the Node.js/Express.js backend via a custom-built RESTful API, exchanging data in JSON format.

3.4 Communications Interfaces

Pothik Bondhu utilizes standard internet protocols to ensure secure and efficient data transfer:

- **HTTP/HTTPS:** Standard web protocols are used for all REST API requests between the client browser and the server.

4. System Features

< >

4.1 System Feature 1

< >

4.1.1 Description and Priority

< >

4.1.2 Stimulus/Response Sequences

< >

4.1.3 Functional Requirements

< >

< >

REQ-1:

REQ-2:

4.2 System Feature 2 (and so on)

5. Other Nonfunctional Requirements

5.1 Performance Requirements

< >

5.2 Safety Requirements

< >

5.3 Security Requirements

< >

5.4 Software Quality Attributes

<>

5.5 Business Rules

<>

6. Other Requirements

<>

Appendix A: Glossary

<>

Appendix B: Analysis Models

<>

Appendix C: To Be Determined List

<>